

writing Python code.

Kivy Language syntax

The first line should be the Kivy header.

```
#:kivy 'version'
<Rule1,Rule2>:
    Definition
    RootClassName:
        definition
```

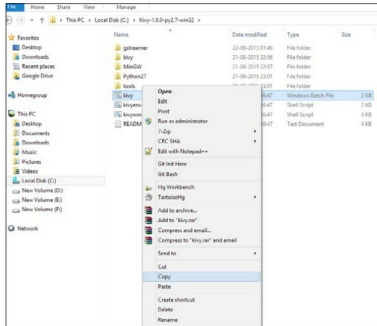


Figure 1: Copy *kivy.bat* file

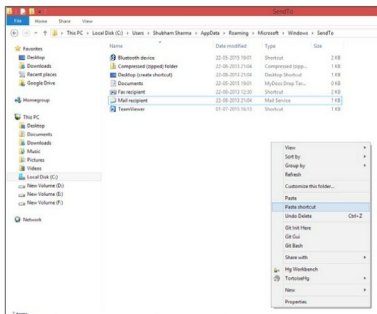


Figure 2: Paste *kivy.bat* as a shortcut in *SendTo*

Here is an example:

1. Create a file *main.py* and add the following code to it:

```
#file name: main.py
fromkivy.app import App
class HelloApp(App):
    pass

if __name__ == '__main__':
    HelloApp().run()
```

2. Now create a new file *hello.kv* and save it with the following code:

```
#File name: hello.kv
#:kivy 1.8.0
Label:
    text: "Hello World"
```

3. Windows users need to right-click *main.py* > *sendTo* > *kivy*. To launch the application, a Linux user needs to run the following code:

```
python main.py
```

The screen given in Figure 4 will be shown.

Widgets

A widget is a user interface element that provides some kind of functionality. It receives the input events. A Kivy widget doesn't necessarily have a visible representation on screen. The widgets are arranged in the form of a tree. There can be a number of child widgets and a *.kv* file can have only one root widget, of which all other widgets are direct or indirect children.

The layout tree can be dynamically modified by the following methods:

<i>add_widget(widget)</i>	This adds a new widget as a child of the current widget
<i>remove_widget(widget)</i>	Removes a widget from the children of the current widget
<i>clear_widget(widget)</i>	Removes all widgets added to the current widget as children

```
>>>root= Widget()
>>>button= Button()
>>>root.add_widget(button)
```